

SentinelNet — Full Implementation Plan

Multi-Agent Adversarial Network Defense Planner

Team size: 4 | Review 1: PEAS, Environment, Algorithms, Q&A (10M) | Review 2: Tools, Execution, Demo, Code Structure (10M)

1. Executive Summary

SentinelNet simulates a company network under attack. One **Red** agent tries to reach a high-value "crown jewel" node by moving through a graph of servers, workstations and databases. Three **Blue** agents — Monitor, Response, and Patch Scheduler — cooperate to detect, contain and recover from the intrusion. The simulation is abstract (no real exploits, no real scanning, no real network traffic) so it is safe, fast to build, and easy to demo.

Four algorithms anchor the four rubric-relevant "Algorithmic Modeling" points:

1. **A*** — Red's attack pathing over a weighted graph
 2. **Bayesian belief update / particle filter** — Monitor's tracking of Red's hidden position from noisy alerts
 3. **Minimax with alpha-beta pruning → Expectimax** — Response's containment decisions under adversarial and stochastic conditions
 4. **CSP with backtracking + MRV + forward checking** — Patch Scheduler's maintenance/patch scheduling under staff and dependency constraints
-

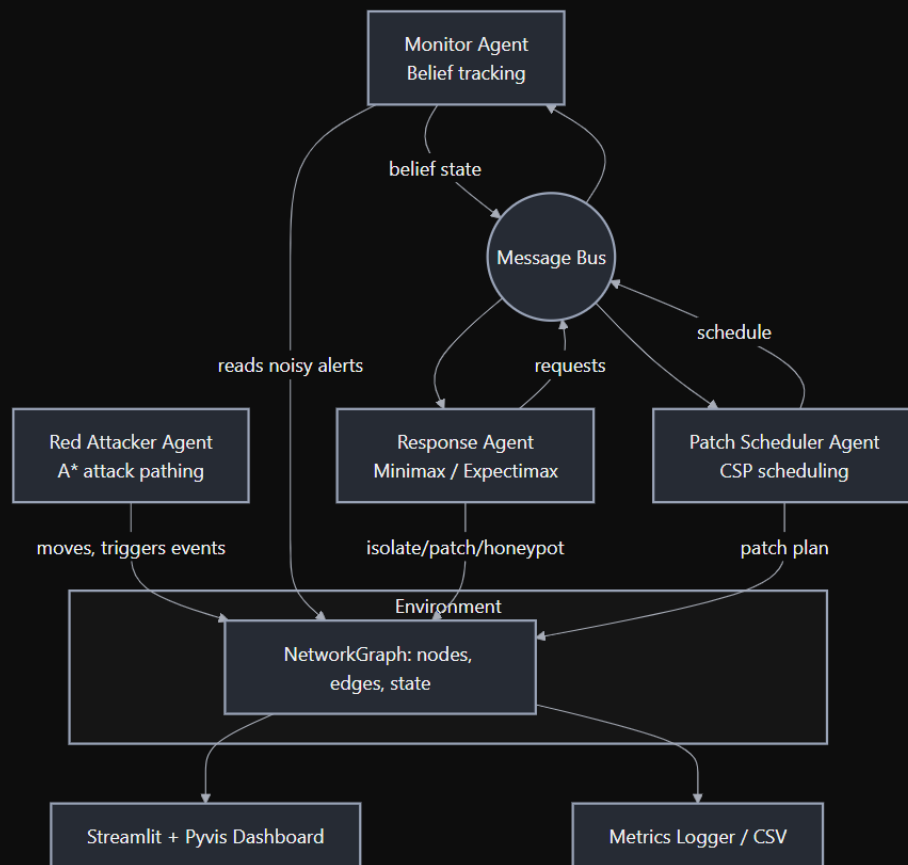
2. PEAS Recap (for Review 1 slide)

Blue Team

- **Performance measure:** compromised-asset count (↓), mean time-to-detect (↓), service uptime (↑), false-positive rate (↓), response cost (↓)
- **Environment:** network graph — nodes have type, value, vulnerability score, patch status; edges have traversal cost/risk
- **Actuators:** patch(node), isolate(node), deploy_honeypot(node), restore(node), reroute(edge), raise_alert(node)
- **Sensors:** IDS alert stream (noisy, probabilistic), traffic volume per edge, vulnerability scan reports, patch-status feed

Environment properties: partially observable, stochastic, sequential, dynamic, discrete, multi-agent (competitive Red↔Blue, cooperative within Blue)

3. System Architecture



4. Repository Structure

```

sentinelnet/
├── README.md
├── requirements.txt
├── config/
│   ├── network_small.yaml      # 20-node demo topology
│   ├── network_medium.yaml     # 100-node topology
│   ├── network_large.yaml      # 500-node scalability test
│   └── scenarios.yaml           # 5 demo scenario definitions
├── sentinelnet/
│   ├── __init__.py
│   └── environment/
│       ├── network_graph.py    # NetworkX wrapper, node/edge state
│       └── events.py           # Alert, ExploitEvent, PatchEvent
├── dataclasses
│   ├── simulator.py            # Tick loop, scenario runner
│   └── agents/
│       ├── base_agent.py
│       ├── red_attacker.py     # A* pathing
│       └── monitor.py          # Belief tracking (Bayesian / particle
filter)
│       ├── response.py         # Minimax / expectimax + alpha-beta
│       └── patch_scheduler.py  # CSP: backtracking + MRV + forward
checking
└── coordination/

```

```

├── message_bus.py          # Pub/sub blackboard for Blue agents
├── baselines/
│   ├── random_defender.py
│   └── greedy_defender.py
├── metrics/
│   └── logger.py          # Compromise rate, TTD, uptime, node/sec
expanded
├── viz/
│   └── dashboard.py       # Streamlit app
├── tests/
│   ├── test_astar.py
│   ├── test_belief_tracking.py
│   ├── test_minimax.py
│   ├── test_csp_scheduler.py
│   └── test_integration_scenarios.py
├── notebooks/
│   └── scalability_benchmark.ipynb

```

5. Tech Stack & Setup

Purpose	Library	Install
Graph representation	NetworkX	<code>pip install networkx</code>
Simulation scaffolding	Plain Python classes (Mesa optional)	<code>pip install mesa (optional)</code>
CSP solving	python-constraint or OR-Tools CP-SAT	<code>pip install python-constraint</code> or <code>pip install ortools</code>
Dashboard	Streamlit + Pyvis	<code>pip install streamlit pyvis</code>
Config	PyYAML	<code>pip install pyyaml</code>
Testing	pytest	<code>pip install pytest</code>
Data/metrics	pandas, matplotlib	<code>pip install pandas matplotlib</code>

requirements.txt:

```

networkx==3.3
python-constraint==1.4.0
streamlit==1.38.0
pyvis==0.3.2
pyyaml==6.0.2
pandas==2.2.2
matplotlib==3.9.0
pytest==8.3.2

```

Setup commands:

```

git clone <repo-url> && cd sentinelnet
python -m venv venv && source venv/bin/activate
pip install -r requirements.txt

```

```
pytest tests/ -v
streamlit run sentinelnet/viz/dashboard.py
```

6. Core Data Model

```
# environment/network_graph.py
from dataclasses import dataclass, field
from enum import Enum
import networkx as nx

class NodeState(Enum):
    SAFE = "safe"
    SUSPECTED = "suspected"
    COMPROMISED = "compromised"
    ISOLATED = "isolated"
    PATCHING = "patching"

@dataclass
class NodeData:
    name: str
    node_type: str          # "workstation" | "server" | "database" |
    "crown_jewel"
    value: int              # asset value, used in performance metric
    vuln_score: float       # 0-1, probability an exploit attempt
    succeeds
    patched: bool = False
    state: NodeState = NodeState.SAFE

class NetworkGraph:
    def __init__(self, config_path: str):
        self.g = nx.Graph()
        self._load_from_yaml(config_path)

    def _load_from_yaml(self, path):
        ... # parse nodes/edges, weight = traversal cost/risk

    def neighbors_with_cost(self, node):
        return [(n, self.g[node][n]["cost"]) for n in
self.g.neighbors(node)]
```

7. Agent Implementations

7.1 Red Attacker — A* pathing

```
# agents/red_attacker.py
import heapq

def a_star(graph, start, goal, heuristic):
    frontier = [(0, start, [start])]
    visited = {}
    nodes_expanded = 0
    while frontier:
        cost, node, path = heapq.heappop(frontier)
        nodes_expanded += 1
        if node == goal:
            return path, cost, nodes_expanded
        if node in visited and visited[node] <= cost:
            continue
        visited[node] = cost
        for nxt, edge_cost in graph.neighbors_with_cost(node):
            new_cost = cost + edge_cost
```

```

        heapq.heappush(frontier, (new_cost + heuristic(nxt, goal), nxt,
path + [nxt]))
    return None, float("inf"), nodes_expanded

```

- **Heuristic:** shortest hop-count to crown jewel ignoring vulnerability (admissible, since real cost $\geq$ hop cost).
- Red recomputes a path whenever Blue isolates a node on its current route (dynamic re-planning).
- Log nodes_expanded per call — this feeds the scalability comparison table.

7.2 Monitor — Belief Tracking

Two options; implement the Bayesian grid version for Review 1, mention particle filter as a stretch goal for larger graphs.

```

# agents/monitor.py
def bayesian_update(belief: dict, alert_node, detection_rate,
false_alarm_rate, graph):
    """
    belief: {node_name: probability}
    P(alert | attacker at node) = detection_rate if alert_node == node else
false_alarm_rate
    """
    new_belief = {}
    for node, p in belief.items():
        likelihood = detection_rate if node == alert_node else
false_alarm_rate
        new_belief[node] = p * likelihood
    total = sum(new_belief.values()) or 1e-9
    return {n: p / total for n, p in new_belief.items()}

def diffuse_belief(belief, graph, stay_prob=0.6):
    """Attacker moves each tick; spread belief mass to neighbors."""
    new_belief = {n: 0.0 for n in belief}
    for node, p in belief.items():
        neighbors = list(graph.g.neighbors(node))
        new_belief[node] += p * stay_prob
        if neighbors:
            share = p * (1 - stay_prob) / len(neighbors)
            for nb in neighbors:
                new_belief[nb] += share
    return new_belief

```

Publish belief on the message bus every tick; Response consumes the top-k most likely nodes as the "threat state" for its game tree.

7.3 Response — Minimax → Expectimax with Alpha-Beta

```

# agents/response.py
def minimax(state, depth, alpha, beta, maximizing, actions_fn, result_fn,
utility_fn):
    if depth == 0 or state.is_terminal():
        return utility_fn(state), None
    best_action = None
    if maximizing: # Blue's turn
        value = float("-inf")
        for action in actions_fn(state, "blue"):
            child = result_fn(state, action, "blue")
            score, _ = minimax(child, depth - 1, alpha, beta, False,
actions_fn, result_fn, utility_fn)
            if score > value:
                value, best_action = score, action

```

```

        alpha = max(alpha, value)
        if alpha >= beta:
            break
    return value, best_action
else: # Red's turn (modeled via belief, or expectimax over exploit
success)
    value = float("inf")
    for action in actions_fn(state, "red"):
        child = result_fn(state, action, "red")
        score, _ = minimax(child, depth - 1, alpha, beta, True,
actions_fn, result_fn, utility_fn)
        if score < value:
            value = score
        beta = min(beta, value)
        if alpha >= beta:
            break
    return value, best_action

```

- Replace the min-node with a **chance node** (expectimax) once you introduce probabilistic exploit success: $E[\text{value}] = \sum(p_{\text{outcome}} * \text{value}(\text{outcome}))$.
- $\text{utility_fn}(\text{state}) = \text{weighted sum of } (\text{uptime}, -\text{compromised_value}, -\text{response_cost})$.
- Depth-limit to 2–3 plies (branching factor = number of defensible nodes) and use the belief distribution from Monitor to restrict which Red actions are considered plausible — this is what keeps the game tree tractable and is a good Q&A talking point ("why depth-limited, why expectimax vs plain minimax").

7.4 Patch Scheduler — CSP

```

# agents/patch_scheduler.py
from constraint import Problem, AllDifferentConstraint

def build_patch_csp(jobs, technicians, time_slots, dependencies):
    problem = Problem()
    for job in jobs:
        problem.addVariable(job.id, time_slots)
    for job in jobs:
        # constraint: job's slot must be within its maintenance window
        problem.addConstraint(lambda s, w=job.window: s in w, (job.id,))
    for (a, b) in dependencies:
        # a must be patched before b
        problem.addConstraint(lambda sa, sb: sa < sb, (a, b))
    for tech, assigned_jobs in technicians.items():
        # no two jobs for the same technician overlap
        problem.addConstraint(AllDifferentConstraint(), assigned_jobs)
    return problem

```

- Implement your own backtracking + **MRV** (minimum remaining values) + **forward checking** by hand for the report even if python-constraint is used for the working demo — graders will ask you to explain the search, not just call a library.
- Variables: each pending patch job. Domain: available time slots. Constraints: maintenance window, dependency ordering, technician non-overlap, "critical node patched before N ticks".

8. Message Bus / Coordination Protocol

```

# coordination/message_bus.py

```

```

class MessageBus:
    def __init__(self):
        self.topics = {"belief": [], "actions": [], "patch_plan": [],
"alerts": []}

    def publish(self, topic, message):
        self.topics[topic].append(message)

    def latest(self, topic):
        return self.topics[topic][-1] if self.topics[topic] else None

```

Protocol per tick:

1. Environment emits alerts (possibly empty/noisy).
2. Monitor reads alerts, publishes updated belief.
3. Response reads belief, runs minimax/expectimax, publishes actions, executes them on the graph.
4. Every N ticks, Patch Scheduler reads outstanding vulnerability list, publishes patch_plan, executes the next scheduled job.

This satisfies the "Multi-Agent Execution & Interaction" rubric line directly — you can point to the bus and say exactly which agent reads/writes which topic.

9. Simulation Engine (Turn Loop)

```

# environment/simulator.py
class Simulator:
    def __init__(self, graph, red, monitor, response, patch_scheduler, bus,
logger):
        ...

    def run(self, max_ticks=200):
        for t in range(max_ticks):
            alert = self.red.step(self.graph)                # A* move, maybe
triggers alert
            self.bus.publish("alerts", alert)
            belief = self.monitor.update(self.bus.latest("alerts"),
self.graph)
            self.bus.publish("belief", belief)
            action = self.response.decide(self.graph, belief)
            self.graph.apply(action)
            if t % self.patch_interval == 0:
                plan =
self.patch_scheduler.schedule(self.graph.pending_jobs())
                self.graph.apply_patch_plan(plan)
                self.logger.record(t, self.graph, action)
                if self.graph.crown_jewel_compromised() or self.red.captured:
                    break
            return self.logger.summary()

```

10. Visualization (Streamlit + Pyvis)

- Left panel: live graph, node color = state (green=safe, yellow=suspected, red=compromised, gray=isolated, blue=patching).
- Right panel: belief distribution bar chart (top-5 most-likely attacker nodes), running metrics (uptime %, compromised count, TTD).
- Controls: scenario selector, play/pause/step, speed slider.

- Re-render the Pyvis graph each tick by rebuilding the HTML and calling `st_pyvis` or `components.html`.

```
# viz/dashboard.py (sketch)
import streamlit as st
from pyvis.network import Network

st.title("SentinelNet - Live Defense Simulation")
scenario = st.selectbox("Scenario", list(scenarios.keys()))
if st.button("Run"):
    sim = build_simulator(scenario)
    placeholder = st.empty()
    for t in range(sim.max_ticks):
        sim.step()
        net = Network(height="600px")
        for node, data in sim.graph.g.nodes(data=True):
            net.add_node(node, color=color_for_state(data["state"]))
        for u, v in sim.graph.g.edges():
            net.add_edge(u, v)
        placeholder.components.html(net.generate_html(), height=620)
```

11. Testing Plan & Demo Scenarios

Unit tests (tests/): A* returns optimal path on a known small graph; belief update sums to 1 and shifts toward the alerting node; minimax picks the provably-best action in a hand-built 2-ply tree; CSP scheduler respects all dependency/window/technician constraints and fails gracefully when over-constrained.

`config/scenarios.yaml` — five demo scenarios:

1. **Baseline:** naive Red vs random Blue defender
2. **Stealthy attacker:** low alert/detection rate → shows belief tracking earning its keep
3. **Multi-entry attack:** two simultaneous Red entry points → tests Response's prioritization
4. **Resource-starved defense:** Blue can only take one action per 2 ticks → tests CSP/scheduling trade-offs
5. **Scalability run:** same scenario on 20 / 100 / 500-node graphs, logging nodes expanded and wall-clock time with and without alpha-beta pruning

Results table to include in the report/demo (fill in after running):

Strategy	Compromise Rate	Mean TTD (ticks)	Uptime %	Avg nodes expanded (A*)
Random defender				
Greedy defender				
Minimax/expectimax defender				

12. Timeline (4-person team, 5 weeks — adjust to your actual review dates)

Week 1 — Foundations (targets Review 1 content)

- All: finalize PEAS, environment properties, and algorithm choices as slides
- Person A: `network_graph.py`, YAML config loader, 20-node demo topology
- Person B: `red_attacker.py` A* implementation + unit tests
- Person C: `monitor.py` Bayesian belief update + unit tests
- Person D: `response.py` skeleton, utility function design, message bus

Week 2 — Core algorithms

- Person B: Red's dynamic re-planning when isolated
- Person C: belief diffusion step; move to particle filter if time allows
- Person D: minimax + alpha-beta, then expectimax variant; unit tests
- Person A: `patch_scheduler.py` CSP model + hand-rolled backtracking/MRV/forward checking

Review 1 checkpoint: PEAS table, environment property justification, algorithm write-ups with complexity, and a working (even if partial) demo of A* + belief tracking.

Week 3 — Integration

- All: wire agents together through `simulator.py` and the message bus
- Person A + D: baselines (random, greedy defenders)
- Person C: metrics logger

Week 4 — Visualization & scenarios

- Person B: Streamlit + Pyvis dashboard
- Person D: implement the 5 demo scenarios in `scenarios.yaml`
- Person A + C: scalability benchmark notebook (20/100/500 nodes)

Week 5 — Polish for Review 2

- All: run full test suite, fix edge cases, record results table
- All: rehearse the live demo end-to-end at least 3 times
- Prepare Q&A answers (Section 13 below)

13. Rubric-to-Deliverable Checklist

Review 1 (10M)

- PEAS table (3M) — Section 2
- Environment & agent analysis (3M) — Section 2 + architecture diagram
- Algorithmic modeling & search strategy (3M) — Section 7, with complexity stated (A*: $O(b^d)$ worst case; minimax+alpha-beta: $O(b^{(d/2)})$; CSP: worst-case exponential, mitigated by MRV/forward checking)
- Q&A readiness (1M) — Section 14

Review 2 (10M)

- Tool/package selection & setup (3M) — Section 5, `working requirements.txt`, clean install demo

- Multi-agent execution & interaction (3M) — Section 8 message bus + live demo showing Monitor→Response→Patch Scheduler handoffs
- Demo quality & testing scenarios (3M) — Section 11, 5 scenarios + results table
- Code structure & scalability (1M) — Section 4 repo layout + Section 11 scalability run (20/100/500 nodes)

14. Likely Q&A and Prepared Answers

- **"Why expectimax instead of plain minimax?"** Because exploit success is probabilistic, not adversarial at that step — Red doesn't control whether an exploit succeeds, chance does. Minimax assumes a rational adversary at every ply; expectimax correctly averages over the exploit's probability distribution.
- **"Why is this partially observable?"** Blue never directly sees Red's location — only noisy alerts. That's why Monitor maintains a probability distribution instead of a single known state.
- **"What's the complexity of your search, and how did you bound it?"** A^* is $O(b^d)$ worst case but the hop-count heuristic keeps it close to optimal-path length in practice; minimax is depth-limited to 2–3 plies and alpha-beta typically cuts the effective branching factor to its square root; CSP backtracking is worst-case exponential in the number of jobs, but MRV + forward checking prune most of that in practice — show the scalability table as evidence.
- **"What happens if Red is smarter (e.g., also plans against Blue's belief)?"** Discuss as a stretch goal: model it as a partially observable stochastic game (POSG) or give Red a simple counter-deception behavior (fake moves toward decoy nodes) — good to mention you're aware of the limitation.
- **"How do you validate the CSP scheduler is correct, not just 'runs'?"** Unit tests assert no constraint violations across randomized job sets, including an intentionally over-constrained case that should fail with a clear reason.
- **"How does this scale?"** Show the 20/100/500-node benchmark: nodes expanded and wall-clock time, with/without alpha-beta pruning, and belief-tracking using top-k pruning instead of tracking every node once the graph is large.

15. Stretch Goals (if time remains)

- Swap the Bayesian grid belief for a particle filter to scale belief tracking past a few hundred nodes.
- Add a second Red strategy (deceptive/decoy-seeking) to strengthen the "competitive" story in the Q&A.
- Formulate Blue's problem as a POMDP and mention (without necessarily fully implementing) a simple lookahead policy over belief state, for extra depth in algorithm discussion.
- Add a cost-of-false-positive term to the utility function and show how it shifts Response's behavior.

16. Team Role Summary

Member	Primary Agent	Primary Algorithm	Secondary Responsibility
A	Environment + Patch Scheduler	CSP (backtracking, MRV, forward checking)	Repo scaffolding, config system
B	Red Attacker	A* search	Streamlit/Pyvis dashboard
C	Monitor	Bayesian belief update / particle filter	Metrics logging, scalability benchmark
D	Response	Minimax → Expectimax + alpha-beta	Message bus, baselines, demo scenarios